

SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 1

1. What Are We Building ?

We are designing a system similar to Ticketmaster, the world's largest ticketing platform that allows users to discover events, view seating, book tickets, make payments and receive digital tickets.



Ticketmaster Overview

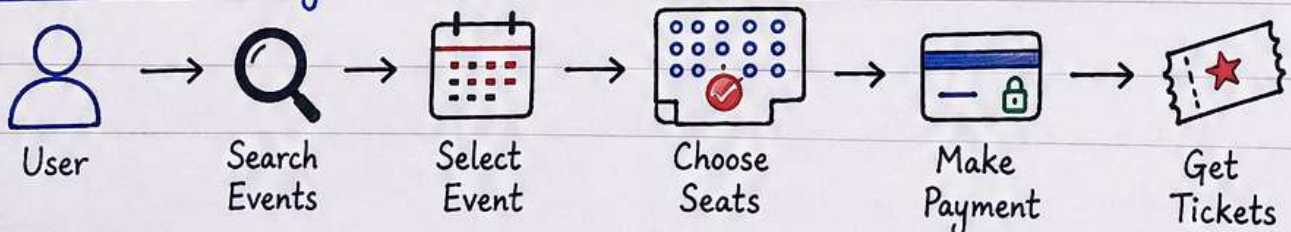
- Connects millions of users with live events.
- Handles huge traffic during popular events.
- Ensures correct seat availability.
- Prevents double booking.
- Provides real-time updates.
- Supports secure payments and ticket delivery.

Ticketmaster in short:

- Events can be concerts, sports, theatre, comedy shows, etc.
- Users search → select seats → pay → get confirmed tickets.
- System must handle massive scale + high contention.
- Real-time seat availability is critical.

Our Goal: Build a highly available, scalable and consistent ticket booking system that can handle millions of users.

Core User Journey:



SYSTEM DESIGN HANDBOOK

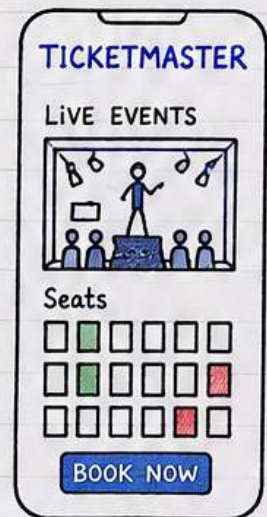
Design Ticketmaster

Page 2

2. Functional Requirements

What should the system be able to do ?

- Users should be able to **browse** and **search** events.
- Users should be able to **view** event details, seat map and ticket pricing.
- Users should be able to **select** seats and **hold** them for a limited time.
- Users should be able to **book** tickets and make **payments**.
- System should **prevent double booking** of the same seat.
- Users should receive **confirmation** and **digital tickets**.
- Admins should be able to **manage** events, venues, seats and pricing.
- System should support **real-time seat availability** updates.



3. Non-Functional Requirements

	High Availability	System should be available 24x7 with o minimal downtime.
	Strong Consistency for Booking	No two users should be able to book the same seat.
	Low Latency	Search results and seat availability should be returned in milliseconds .
	Read Heavy Workload	System should handle millions of read requests per second.
	Massive Traffic Spikes	System should handle sudden spikes during popular event ticket sales.
	Security	Protect user data, prevent fraud, secure payments and tickets.
	Scalability	System should be horizontally scalable to support millions of users.
	Fault Tolerance	System should gracefully handle failures and recover automatically.



Key Takeaway :

Ticketmaster is a read-heavy, high-traffic system where the biggest challenge is ensuring consistency and preventing double booking during massive traffic spikes.

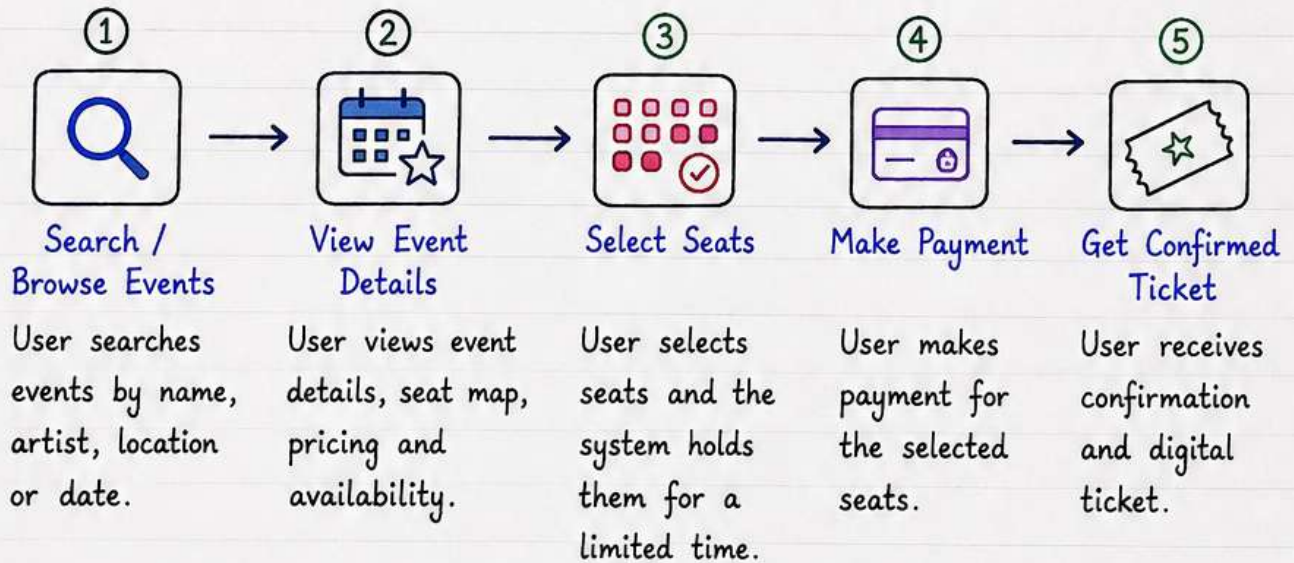
SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 3

4. Core User Journey

Let's walk through the main journey of a user booking a ticket.



★ Key Points in Journey:

- Seats are **held (not confirmed)** for a short time to complete payment.
- If payment fails or timeout happens, seats are **released back**.
- High traffic and concurrency happen mostly during **seat selection & payment**.
- System must **prevent double booking** at all costs.

5. Out of Scope (For Now)

- × Resale marketplace
- × Concert recommendations / ML features
- × Fraud detection system
- × Marketing / notifications
- × Event creation by organizers
- × Physical ticket delivery logistics



Note:

We are focusing only on the core ticket booking flow and handling massive traffic + contention during high demand events.

6. Assumptions

- System will be accessed by web and mobile applications.
- Majority of traffic is read-heavy (search, view events, seat map).
- Write operations (booking) are limited but highly critical.
- Popular events can get millions of concurrent users.
- All times mentioned later are in IST (Indian Standard Time).



SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 4

7. Capacity Estimation (Back-of-the-Envelope)

Let's estimate the scale for our system.

① Users

- Assume 50M registered users
- 15M MAU (Monthly Active Users)

② Events

- 1M events listed per year
- 10K active events per day (globally across all cities)

④ Read / Write Ratio

- Read Heavy System
- Read : Write $\approx 1000 : 1$

③ Traffic

- Normal days: $\sim 2K$ requests/second
- Popular events: up to 100K – 200K requests/second
- Peak (Big events): up to 1M requests/second

⑤ Bookings (Write)

- Assume 100K tickets sold per popular event in few hours
- Avg write: 20 – 50 bookings/second during peak

⑥ Storage Estimation

Data	Estimation
Users	50M
Events	1M / year
Tickets	Assume 100M tickets
Bookings	$\sim 500M$ / year
Avg data per booking	~ 1 KB
Total Storage / year	~ 500 GB
With Indexes & replicas	$\sim 2 - 3$ TB

⑦ Peak Scenario Example

A very popular concert goes live.
10 Million users try to book at the same time.

Target:

- ★ Keep system available
- ★ No double booking
- ★ Fast search & seat selection
- ★ Real-time seat updates
- ★ Successful payments

★ Key Takeaways

- System is **read-heavy** with occasional massive write spikes.
- Search and seat availability must be **extremely fast**.
- Booking flow must be **highly consistent** (no overbooking).
- Architecture must handle **massive scale** and sudden spikes gracefully.
- **Real-time updates** and **caching** will be critical.



Scale Up!

SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 5

8. Core Entities & Database Design

Below are the core entities in our system and their relationships.

① User

- Customers who browse, book and buy tickets.
- Fields: id, name, email, phone, password, created_at

② Event

- An event like a concert, show, sports match, etc.
- Fields: id, name, description, start_time, end_time, venue_id

③ Venue

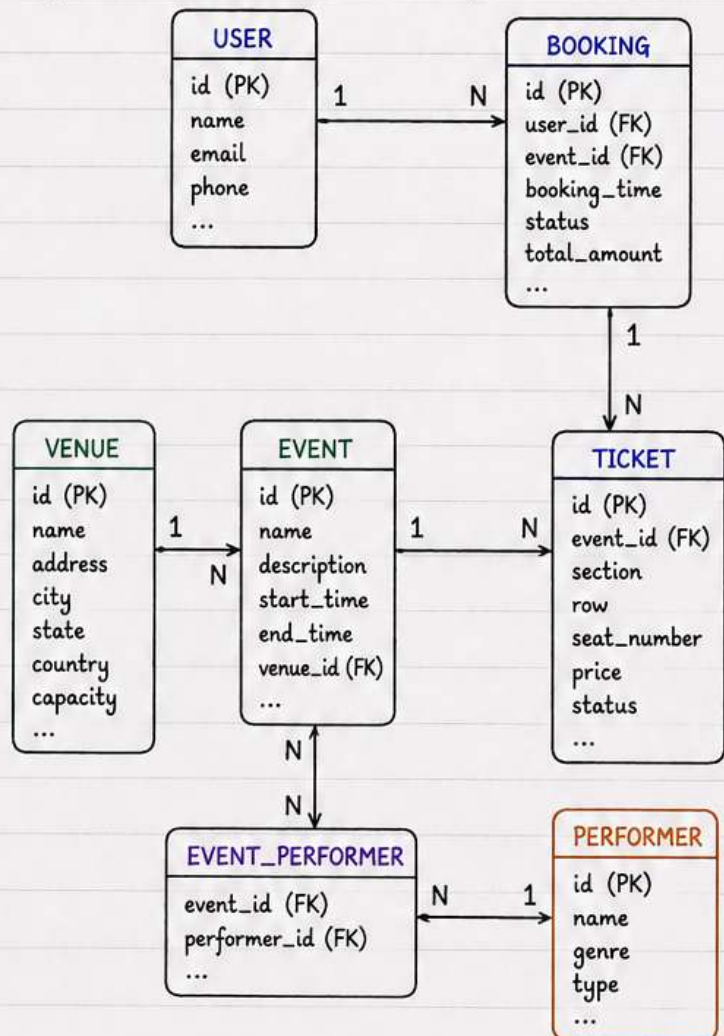
- Location where the event happens.
- Fields: id, name, address, city, state, country, capacity

④ Performer

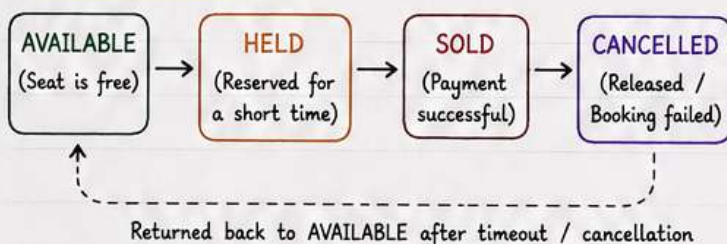
- Artist / Band / Team performing in the event.
- Fields: id, name, genre, type

⑤ Ticket

- Represents a single seat (or ticket unit).
- Fields: id, event_id, section, row, seat_number, price, status



Ticket Status Flow



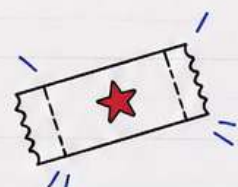
Key Points

- ★ One user can have many bookings.
- ★ One booking can contain many tickets.
- ★ One event has many tickets.
- ★ One venue hosts many events.
- ★ An event can have many performers and a performer can be part of many events (Many-to-Many).



Why this design?

- Separates concerns (User, Event, Ticket, Venue, Performer).
- Many-to-many between **Event** and **Performer** using a junction table.
- Ticket entity stores real seat information and status.
- Easy to scale reads (events, search) separately from writes (bookings).



SYSTEM DESIGN HANDBOOK

Design Ticketmaster

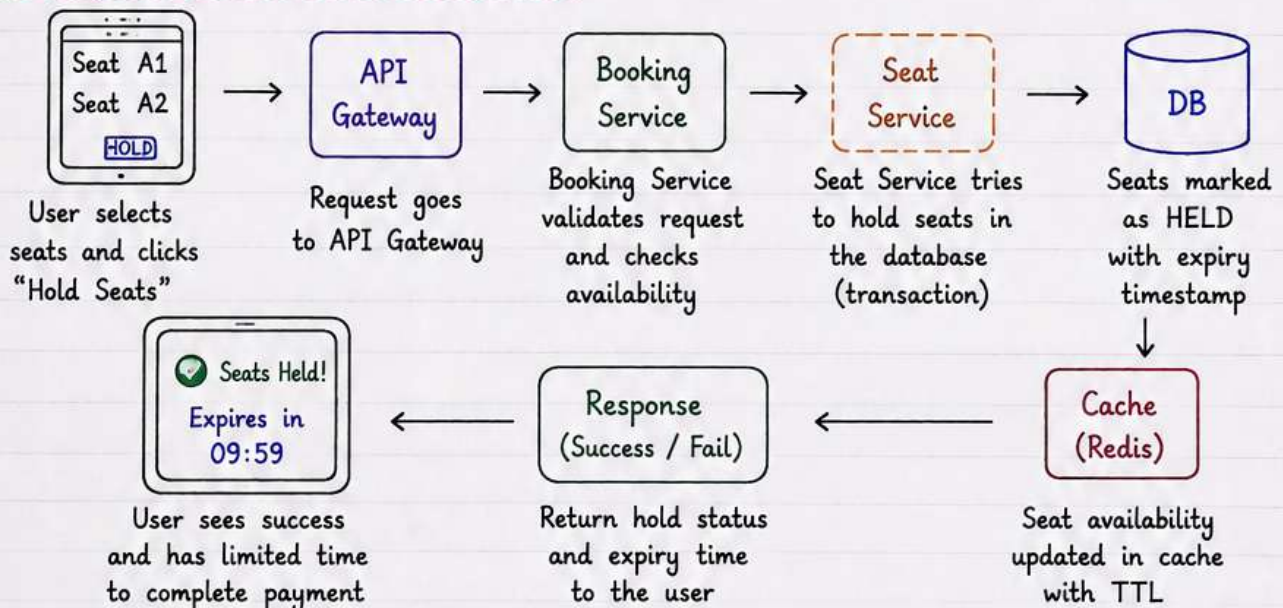
Page 6

9. API DESIGN (Core APIs)

Below are the important APIs for our system.

API	Method	Description	Auth
Get Event Details	GET	Get details of a specific event (including seat map)	No
Search Events	GET	Search events by city, artist, date, category, etc.	No
Get Seat Map	GET	Get seat map and availability for an event	No
Hold / Reserve Seats	POST	Hold selected seats for a short time (e.g., 10 minutes)	Yes
Confirm Booking	POST	Confirm booking after successful payment	Yes
Cancel Booking	POST	Cancel a confirmed booking	Yes
Get Booking Details	GET	Get details of a user's booking	Yes
Webhook (Payment)	POST	Payment gateway notifies payment success / failure	System

Example: Hold / Reserve Seats Flow



★ Key Takeaways

- APIs are designed keeping in mind **read-heavy** (GET) and **write-critical** (POST) nature.
- Hold API** is crucial to prevent double booking and provide a smooth user experience.
- Webhook** helps in handling **async payment** updates from payment gateway.



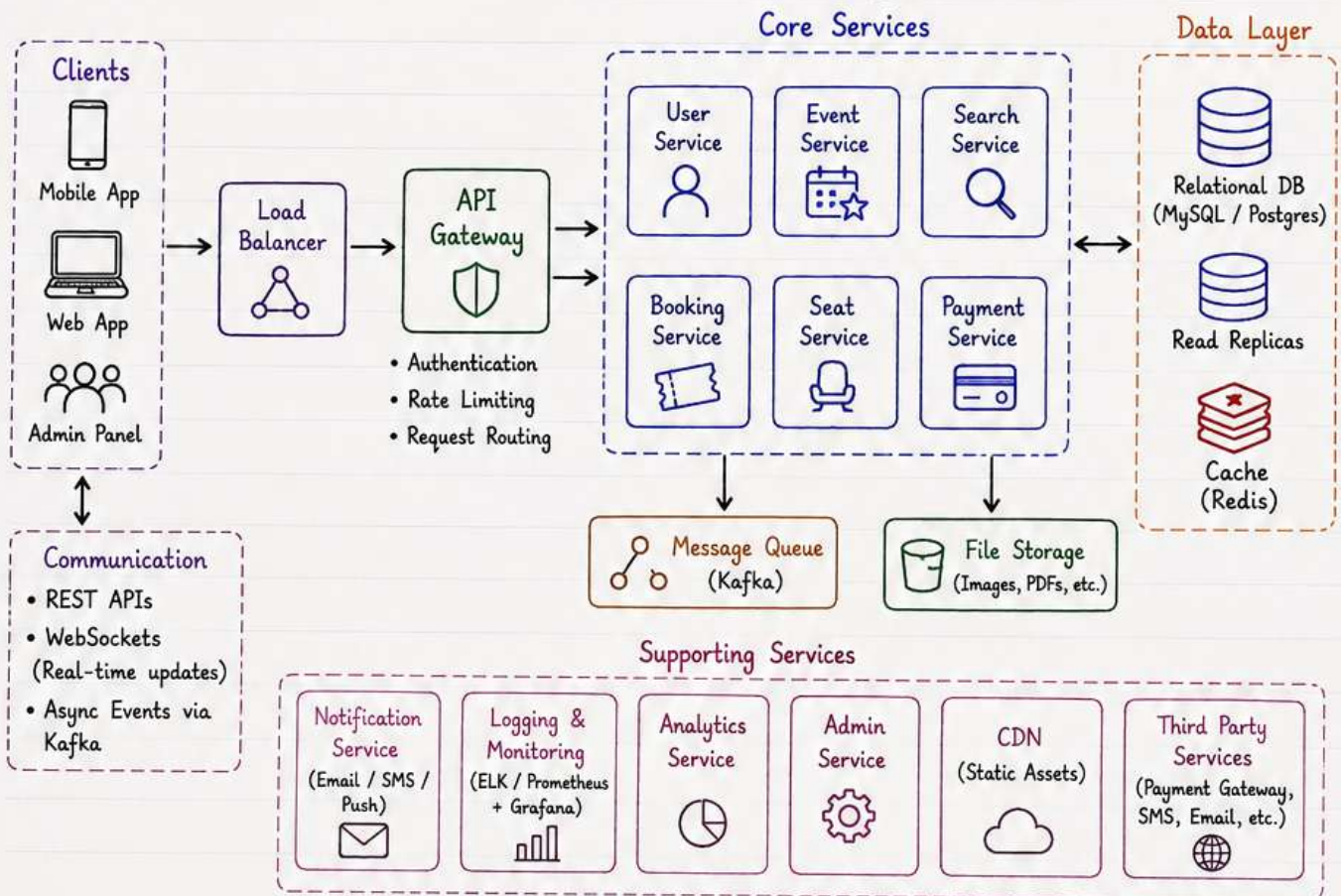
SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 7

10. High Level Architecture

This is the high level architecture of our Ticketmaster system.



How it works?

- 1 Client sends requests from Web / Mobile App.
- 2 Load Balancer distributes traffic to API Gateway.
- 3 API Gateway authenticates, rate limits and routes requests to appropriate services.
- 4 Services read/write data from databases and cache.
- 5 For heavy write operations like booking, events are published to Kafka for async processing.
- 6 Supporting services handle notifications, logging, monitoring and analytics.
- 7 CDN delivers static content (images, JS, CSS) globally with low latency.

Why this architecture?

- ★ Microservices allow independent scaling.
- ★ API Gateway protects backend services and provides a single entry point.
- ★ Databases + Cache handle read-heavy workload.
- ★ Kafka decouples services and handles spikes.
- ★ CDN reduces latency for static content worldwide.
- ★ Read replicas improve read scalability.
- ★ Caching (Redis) speeds up hot data access like events, seats, etc.



Key Takeaways

- Architecture is **microservices** based for scalability and maintainability.
- Read-heavy operations are optimized using **cache**, **read replicas** and **CDN**.
- Booking flow (write-heavy) is handled using **transactions + Kafka** for reliability.
- System is designed to handle **massive traffic spikes** during popular events.



Built to Scale!

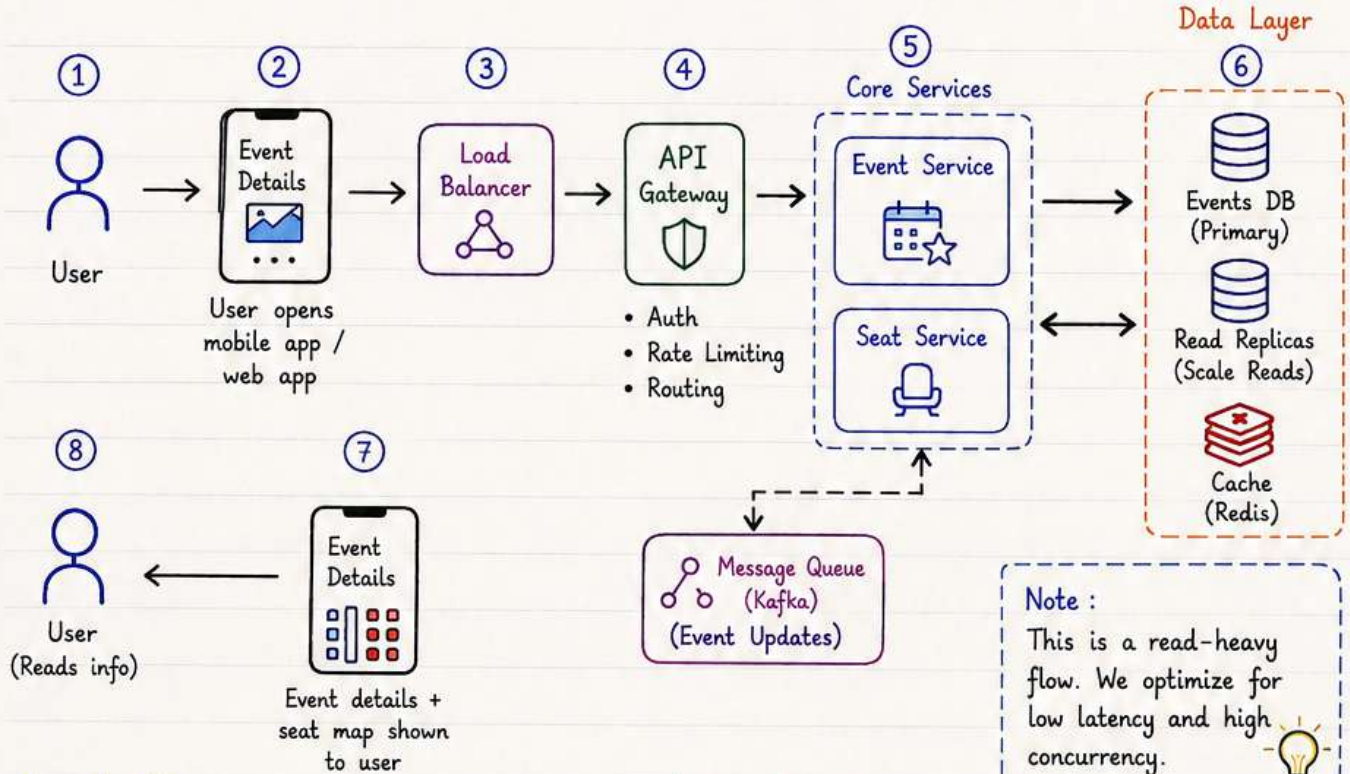
SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 8

11. View Event Flow (Read Path)

Let's see how a user views an event and its details + seat map.



Step-by-Step

- 1 User wants to view an event.
- 2 Request is sent from mobile/web app.
- 3 Load Balancer distributes traffic across available API Gateways.
- 4 API Gateway authenticates user, applies rate limits and routes request to Event Service.
- 5 Event Service fetches event details, Seat Service fetches seat map and availability.
- 6 Data is read from Cache (Redis). If miss, it goes to Read Replicas. If still not available, it goes to Primary DB.
- 7 Response is aggregated and sent back to the app.
- 8 User sees event details and seat map instantly.

Key Points

- ★ Cache (Redis) stores hot data like events, seat maps, pricing for faster reads.
- ★ Read Replicas handle high read traffic and reduce load on Primary DB.
- ★ Seat map is a heavy read. We cache by event_id + section.
- ★ Seat availability is real-time and updated via events from Kafka.
- ★ API Gateway protects services and maintains a single entry point.

Example Read Flow (Numbers)

- 10,000 users view the same popular event.
- Cache hit ratio: 90% → Fast response.
- 9,000 requests served from Cache.
- 1,000 requests go to Read Replicas.
- Very few hit the Primary DB.



★ Key Takeaways

- Read path must be extremely fast and scalable.
- Use Cache + Read Replicas + CDN to handle massive read traffic.
- Event and Seat data are fetched independently and aggregated.
- System is designed for millions of concurrent views without affecting bookings.



SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 9

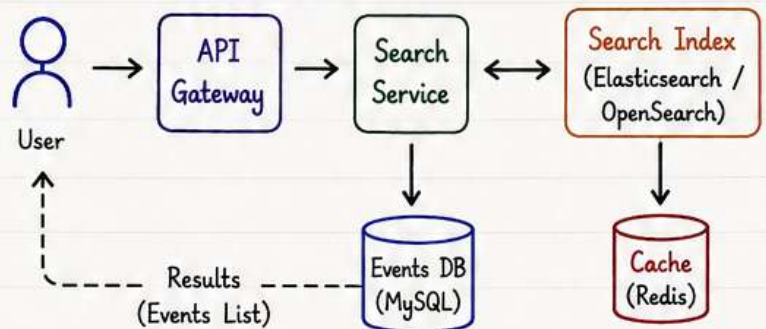
12. Event Search Architecture (Read Path)

Users search for events by keyword, city, date, artist, category, venue, etc.
We need fast, relevant results with filters, sorting and pagination.

Search Requirements

- ★ Keyword search (event name, description)
- ★ Filter by city, date, artist, category, venue, price, etc.
- ★ Sort by relevance / date / popularity
- ★ Pagination
- ★ Autocomplete suggestions
- ★ High availability & low latency

Search Flow



Search Query Example

```
GET /search/events?
q=arijit singh&city=mumbai&date=2025-06-01
&category=music&sort=date
&page=1&size=20
```

Filters & Sorting Examples

- ★ city = Mumbai, Delhi, Bangalore
- ★ date >= 2025-06-01
- ★ artist = "Arijit Singh"
- ★ category = Music, Sports, Comedy
- ★ sort = relevance | date | popularity | price_asc
- ★ page = 1, size = 20



Search Components

Component	Purpose	Technology
Search Service	Handles search requests, builds query, aggregates results	Node.js / Java / Go
Search Index	Stores indexed events for fast full-text search	Elasticsearch / OpenSearch
Cache	Caches popular search queries & results	Redis
Events DB	Source of truth for events data	MySQL (Read Replicas)
Autocomplete	Provides suggestions for keywords, artists, venues	Redis / ES Suggest API

Indexing Strategy

- ★ Create inverted index for fields like name, description, venue, artist, city.
- ★ Use analyzed text for full text search.
- ★ Keyword fields for exact match (city, category, artist_id).
- ★ Geo-index for location based search.
- ★ Update index in real-time using change data capture or events.



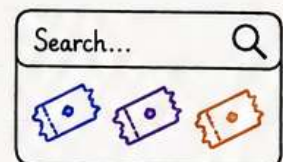
Why Elasticsearch / OpenSearch ?

- Fast full-text search
- Scalable and distributed
- Built-in relevancy scoring
- Aggregations & filtering
- Handles millions of documents



Key Takeaways

- Search is **read-heavy** → Use **Search Index** (Elasticsearch/OpenSearch).
- **Cache** popular queries to reduce load on search cluster.
- Keep index updated in **near real-time** using events (Kafka).
- Design for **filters, sorting, pagination** and **autocomplete**.
- **Scale** search cluster horizontally to handle peak traffic.



SYSTEM DESIGN HANDBOOK

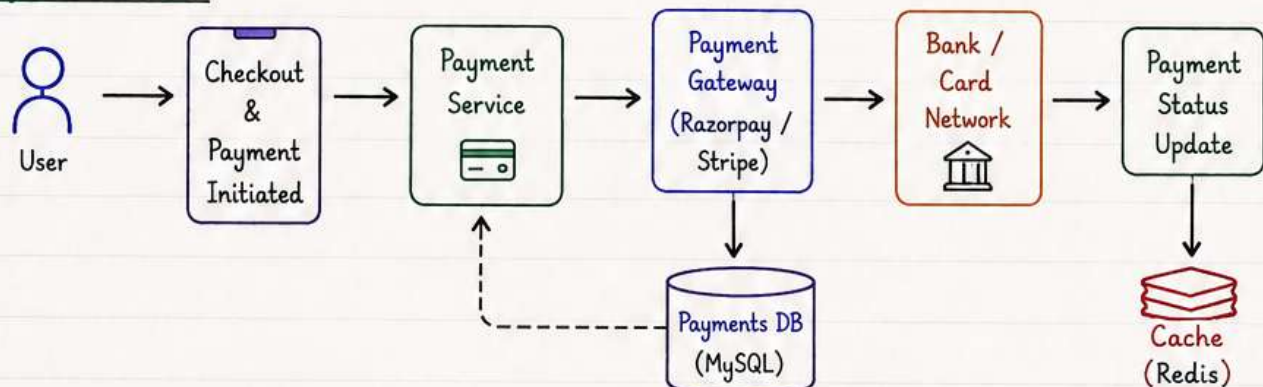
Design Ticketmaster

Page 10

13. Payment Processing Architecture (Write Path)

Let's see how payments are processed securely and reliably.

High Level Flow



Step-by-Step Write Flow

- 1 User selects seats and proceeds to checkout.
- 2 Payment Service creates an order and saves it as PENDING in Payments DB.
- 3 User is redirected to Payment Gateway.
- 4 User completes payment on Gateway page.
- 5 Gateway sends webhook / callback to Payment Service.
- 6 Payment Service updates status to SUCCESS / FAILED.
- 7 Cache is updated and booking is confirmed on success.

Key Tables

Payments Table

- id (PK)
- order_id (FK)
- user_id (FK)
- amount
- currency
- status
- gateway_txn_id
- created_at
- updated_at
- ...

Orders Table

- id (PK)
- user_id (FK)
- event_id (FK)
- amount
- status (PENDING / SUCCESS / FAILED)
- created_at
- updated_at
- ...

Payment Status

- PENDING
- SUCCESS ✓
- FAILED ✗
- REFUNDED

Payment Gateway Responsibilities

- ★ Secure card / UPI / Wallet payments
- ★ Fraud detection & validation
- ★ 3D Secure Authentication
- ★ Transaction response (Success / Fail)
- ★ Webhook / Callback for final status

Reliability & Consistency

- ★ Use idempotency keys to avoid duplicate payments.
- ★ All write operations are done in DB (source of truth).
- ★ Use webhooks for async payment status updates.
- ★ Retry failed webhook events.

Security Best Practices

- ★ Never store card details.
- ★ Use tokens from Payment Gateway.
- ★ Verify webhook signatures.
- ★ Use HTTPS everywhere.
- ★ Monitor and alert on failures.

Key Takeaways



- Payments are write-heavy and require strong consistency.
- Use a reliable Payment Gateway & Webhooks for final status.
- Update DB as source of truth and Cache for fast reads.
- Handle failures, retries and idempotency for reliability.
- Security and monitoring are critical for payments.



Great events are experienced once,
but great systems make them happen every time.



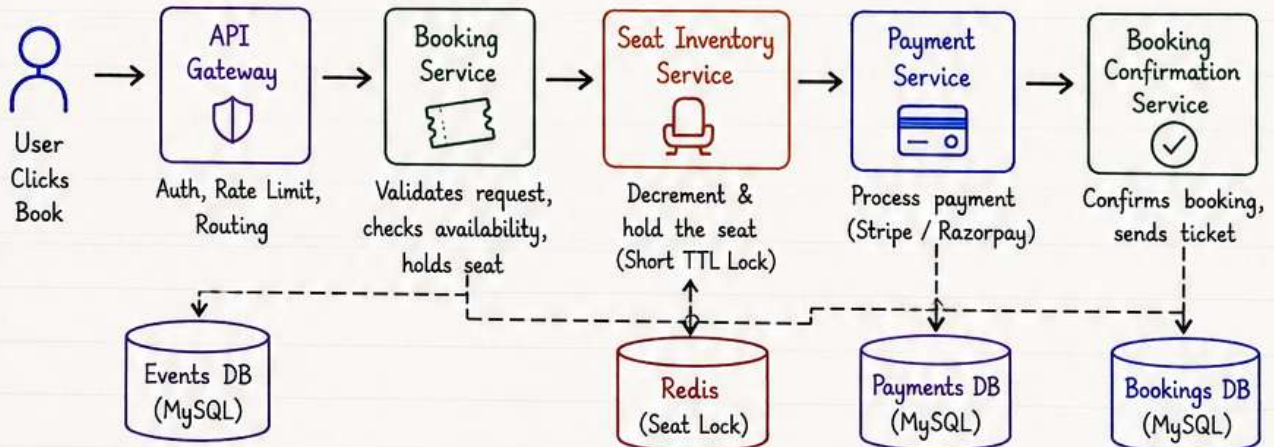
SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 11

14. Seat Reservation Architecture (Write Path)

Seat booking is the most critical part. We must prevent double booking, maintain availability and ensure consistency.



Step-by-Step Write Flow

- 1 User selects seats and clicks 'Book Now'.
- 2 Booking Service checks seat availability with Seat Inventory Service.
- 3 Seats are held in Redis with short TTL (e.g., 5 minutes).
- 4 Payment Service processes the payment.
- 5 On success, Booking is saved in Bookings DB and seats are confirmed.
- 6 Ticket (QR / Barcode) is generated & sent to user.
- 7 If payment fails or times out, seat lock expires and seat becomes available again.

Concurrency Control Mechanisms

- ★ Redis distributed lock on `seat_id` with short TTL.
- ★ Atomic decrement of `seat_count` in DB.
- ★ Optimistic locking (version field) in Inventory DB.
- ★ Idempotency key for booking requests.
- ★ Queue based confirmation to handle retries safely.

Why Redis Lock?

- Extremely fast
- Supports TTL.
- Prevents double booking
- Distributed lock across multiple servers
- Auto release after expiration

Seat Inventory Design

seats (Table)

- `id` (PK)
- `event_id` (FK)
- `section_id`
- `row_no`
- `seat_no`
- `price`
- `status` (AVAILABLE / HELD / BOOKED)
- `version`
- `updated_at`

Status Meaning

- AVAILABLE → Seat is open
- HELD → Temporarily locked
- BOOKED → Confirmed

We update status using atomic operations to avoid race conditions.



Failure Handling

- Payment failed → Release seat lock, mark as AVAILABLE.
- Payment timeout → Lock expires, seat auto released.
- Service down → Retry with exponential backoff.
- Duplicate booking → Prevented using idempotency key.
- DB failure → Use replicas & backups.



Always release locks in case of failure.



Key Takeaways

- Seat booking is write-heavy and highly concurrent.
- Redis lock + Atomic operations ensure no double booking.
- Short TTL locks + Idempotency keys handle failures.
- Queue based confirmations make system reliable.
- Consistency + Speed + Scalability = Successful booking experience.



Great systems handle millions of users,
but unforgettable experiences win hearts.



SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 12

15. Scalability, Availability & Other Considerations

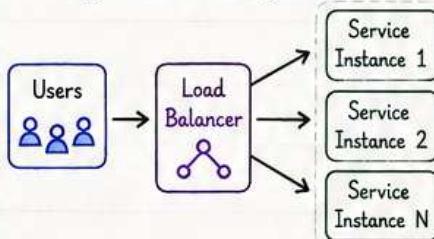
Ticketmaster must handle huge traffic spikes during popular events.

Scalability Strategies

- ★ **Horizontal Scaling:** Add more instances of services behind Load Balancer.
- ★ **Auto Scaling:** Scale services based on CPU / RPS metrics.
- ★ **Database Scaling:** Read Replicas for read heavy workload.
- ★ **Caching:** Use Redis for seat map, events, pricing, and sessions.
- ★ **CDN:** Serve static content (images, JS, CSS) via CDN.
- ★ **Async Processing:** Use Kafka for emails, notifications, analytics.

High Availability

- Deploy services across multiple Availability Zones.
- Health checks & auto recovery.
- Multi-region deployment for disaster recovery.
- Database backups & point-in-time recovery.
- Use Load Balancer to distribute traffic across healthy instances.



Other Considerations

- **Rate Limiting:** Prevent abuse & scalping.
- **Security:** Protect APIs, prevent SQL Injection, XSS.
- **Monitoring:** Track logs, errors, latency, RPS.
- **Alerts:** Setup alerts for failures, high latency, high error rate.
- **Capacity Planning:** Plan for peak events.
- **Cost Optimization:** Right-size instances & use caching.

Key Metrics to Monitor

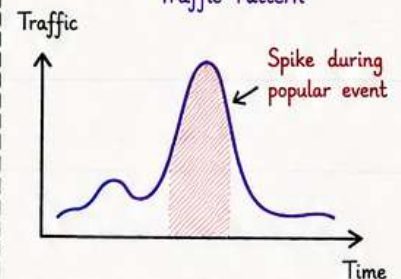
- RPS (Requests per second)
- Error Rate (%)
- Latency (p95, p99)
- Seat Lock Success Rate
- Payment Success Rate
- Infrastructure CPU / Memory
- Queue Lag (Kafka)
- DB Connections



Handling Traffic Spike (Popular Events)

- ① Users enter waiting room.
- ② Queue system assigns random place in line.
- ③ Users are served in order (fair & controlled).
- ④ Rate limiting to prevent bots.
- ⑤ Cache popular data (events, seat map).
- ⑥ Auto scale services & databases.

Traffic Pattern

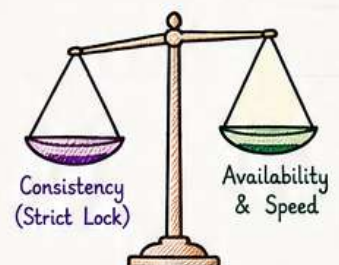


Tech Stack (Example)

Layer	Technology
Frontend	React / Next.js
Backend	Node.js / Java / Go
API Gateway	Kong / Nginx
Database	MySQL (Primary + Read Replicas)
Cache	Redis
Message Queue	Kafka
Search	Elasticesarch / OpenSearch
CDN	CloudFront / Cloudflare
Cloud	AWS / GCP / Azure
Monitoring	Prometheus + Grafana
Logging	ELK Stack / Loki

Trade-offs

- ★ Strong consistency vs Better availability (We choose availability for seat locks)
- ★ Caching improves speed but may show stale data.
- ★ Locking seats strictly prevents double booking but reduces availability.
- ★ More replicas improve read performance but increase cost.
- ★ Queue system adds latency but ensures fairness.



Key Takeaways

- Break down the system into clear services.
- Design for high concurrency, scalability & reliability.
- Use caching, queues, locks & replication wisely.
- Monitor everything and plan for failures.
- User experience (queue, fair access) is as important as performance.



Great systems don't just sell tickets, they create unforgettable experiences.



SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 13

16. Additional Features & Non-Functional Requirements

Let's cover important features and quality attributes.

Additional Features

- ★ User Accounts & Profiles (history, favorites, saved cards)
- ★ Event Recommendations
- ★ Waitlist & Notifications
- ★ Dynamic Pricing
- ★ Group Bookings
- ★ Gift Cards & Vouchers
- ★ Multi-language & Multi-currency
- ★ Admin Dashboard (analytics, reports, user & event mgmt.)

Non-Functional Requirements

- High Availability : 99.99% uptime
- Scalability : Handle millions of concurrent users
- Low Latency : Fast search & booking < 200ms
- Consistency : No double booking
- Durability : No data loss
- Security : Secure payments & user data
- Maintainability : Easy to deploy & monitor
- Usability : Smooth & simple user experience

Quality Pillars



Scalability



Reliability



Security



Performance



Usability

17. Monitoring, Logging & Alerting



Monitoring → Track system health, servers, APIs, DB, queues.



Logging → Centralized logs (ELK Stack) for debugging & audit.



Alerting → Set alerts for high latency, failures, quota breaches.

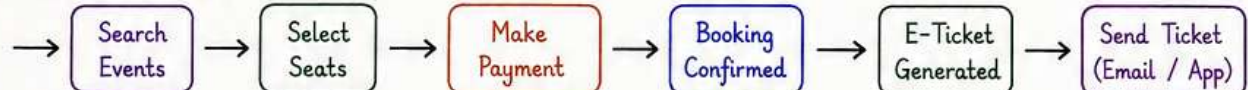
Tools (Example)

- ★ Monitoring : Prometheus + Grafana
- ★ Logging : ELK Stack (Elasticsearch, Logstash, Kibana)
- ★ Alerting : PagerDuty / Opsgenie
- ★ APM : New Relic / Datadog

18. Security Considerations

- ✓ HTTPS everywhere
- ✓ Rate limiting & DDoS protection
- ✓ Input validation & sanitization
- ✓ Secure payment processing (PCI-DSS)
- ✓ Token based authentication (JWT)
- ✓ Secrets management (Vault)
- ✓ Data encryption at rest & in transit

19. Data Flow Summary



All actions are logged, monitored and stored for reliability & analytics.



20. Key Takeaways



- Ticketmaster handles massive scale and high traffic.
- System is divided into small, independent services.
- Seat inventory & payments need strict consistency.
- Caching, queues & CDNs ensure speed and reliability.
- Monitoring, security & scalability are critical.
- Good user experience + Fairness = Happy users!

21. Interview Pointers

- ✓ Why microservices over monolith?
- ✓ How do you prevent double booking?
- ✓ How would you handle a traffic spike?
- ✓ How to design seat locking mechanism?
- ✓ How would you ensure high availability?
- ✓ What are trade-offs in this design?



Great systems are built with clarity, reliability and scale in mind.
Design smart. Build better. Scale fearlessly.



SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 14

22. Capacity Estimation (Back-of-the-Envelope)

Let's do rough calculations for a big event.

Assumptions (Example)

- ★ Event: Coldplay Concert
- ★ Venue Capacity: 100,000 seats
- ★ Tickets per user: 4 (avg)
- ★ Daily Active Users (during sale): 2 Million
- ★ Active sale window: 2 hours
- ★ Read : Write ratio: 90% : 10%
- ★ Average response size: 15 KB

Traffic Estimation

- Total users in 2 hours = 2,000,000
- Peak concurrency (at once) $\approx$ 100,000 users
- Requests per user (avg) = 20
- Total requests in 2 hours = 40,000,000
- Requests per second (avg) = $40,000,000 / 7200 \approx 5,555$ RPS
- Peak RPS (10x of avg) $\approx$ 55,000 RPS

Data Storage Estimation

- ★ Users: 50 Million
- ★ Events: 1 Million / year
- ★ Bookings: 100 Million / year
- ★ Tickets: 400 Million / year
- ★ Average ticket size: 2 KB

Total data / year $\approx$
 $400M \times 2 \text{ KB} = 800 \text{ GB}$
 (Excluding indexes & backups)

High Level Capacity Planning



Application Servers

- Each server can handle $\sim$ 500 RPS
- Required servers (peak): $55,000 / 500 = 110$
- Add 30% buffer $\rightarrow \sim$ 150 servers



Database (Read)

- Read heavy $\rightarrow$ Use Read Replicas
- 1 Primary + 5 Replica (min)
- Scale replicas based on traffic.



Database (Write)

- Writes are less but critical
- 1 Primary (with high IOPS)
- Use partitioning for Bookings & Tickets



Cache (Redis)

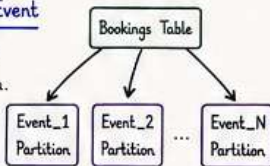
- Cache seat inventory & event details.
- Hit ratio target: $> 90\%$

23. Database Partitioning (Example)

Partition large tables to handle scale.

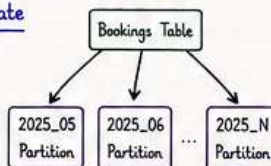
Partitioning by Event

- Each event has its own partition.
- Improves write performance.



Alternative: By Date

- Partition by date (e.g., month).
- Helps in archival & cleanup.



24. Caching Strategies

- ★ Cache seat inventory (short TTL)
- ★ Cache event & venue details
- ★ Cache popular events list
- ★ Cache user sessions (Redis)
- ★ Use CDN for static content
- ★ Invalidate cache on booking

25. System Bottlenecks & How to Handle

Bottleneck	Why it happens	How to Handle
Traffic Spike	Millions of users at the same time	Auto-scaling, Load Balancer, CDN, Queue system
Seat Locking	High contention on popular seats	Redis distributed lock, Short TTL, Optimistic locking
DB Overload	Too many read/write queries	Read replicas, Indexing, Partitioning, Caching
Slow Response	High latency from DB / APIs	Caching, Optimize queries, Async processing
Notification Delay	High volume of emails/SMS	Message queue, Retry mechanism, Rate limiting

26. Future Enhancements

- ✓ AI based recommendations
- ✓ Dynamic pricing
- ✓ User loyalty program
- ✓ Real-time seat suggestions
- ✓ Mobile app with offline mode
- ✓ Blockchain based ticketing



Final Thoughts

Designing a system like Ticketmaster is all about balancing **Scale**, **Consistency**, **Reliability** and **Performance**. Focus on user experience and handle failures gracefully.



A great system creates unforgettable experiences.
 Keep learning. Keep building.

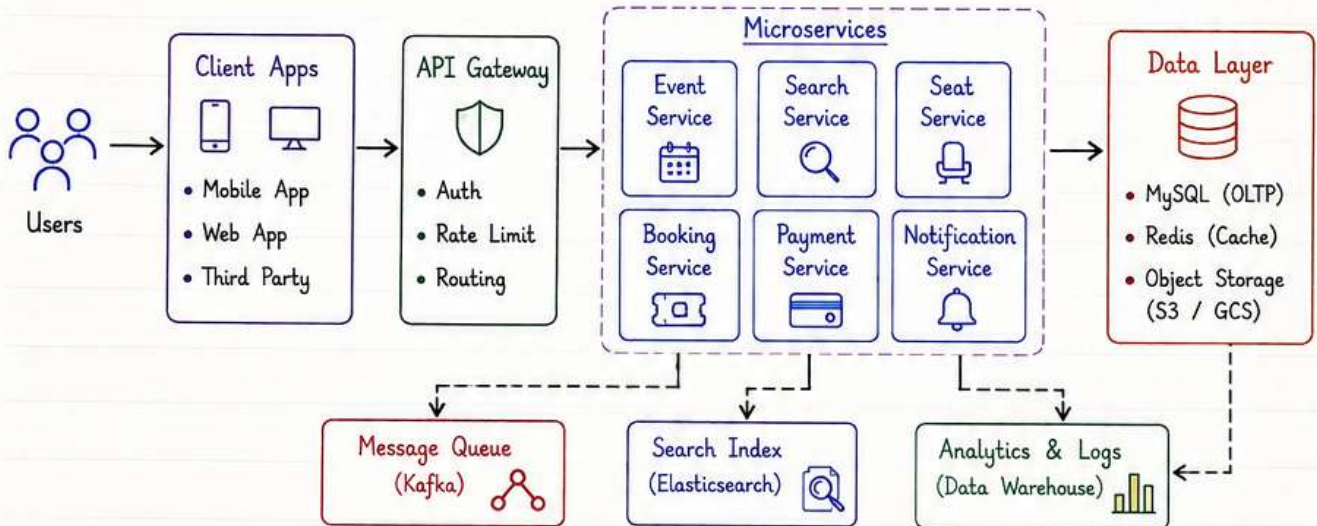


SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 15

27. Putting It All Together (High Level Architecture)



28. Trade-offs in Ticketmaster Design

Area	Trade-off	Why we chose this
 Consistency vs Availability	Strong consistency for seat inventory (Redis Lock)	Prevents double booking, ensures trust.
 Performance vs Cost	Use of CDN, Caching, Read replicas	High performance with optimized infrastructure cost.
 Scalability vs Complexity	Microservices + Async Processing	Adds complexity but handles massive scale easily.
 Storage vs Speed	Store seat map in Redis (volatile)	Fast access during booking, rebuild from DB if needed.
 Cost vs Reliability	Use managed services where possible	Reduces operational overhead & improves reliability.

29. Key Learnings

- ★ Break down big problem into small, independent services.
- ★ Identify critical flows (search, seat lock, payment).
- ★ Ensure consistency where business demands.
- ★ Use caching, queues & CDNs to handle scale.
- ★ Design for failures, retries and monitoring from day one.
- ★ Focus on user experience at every step.



30. Real World Numbers (Approx.)

- Millions of events across the globe.
- Peak traffic during high demand events can be 100x normal.
- Millions of seat locks per second.
- Thousands of payments per second.
- System should support 99.99%+ availability.

31. What to Do Next?

- ✓ Try designing similar systems: BookMyShow, Uber, Zomato.
- ✓ Dive deeper into each microservice.
- ✓ Learn Redis Locking, Kafka, Elasticsearch in depth.
- ✓ Practice capacity estimation for more products.
- ✓ Build small projects to strengthen concepts.



Summary

Ticketmaster is a perfect example of how System Design concepts come together to solve a real-world, high scale, mission-critical problem. Great design is not about using every technology, but using the RIGHT technology in the RIGHT place for the RIGHT reason.



Design with clarity. Build with purpose.
Scale with confidence. Deliver unforgettable experiences.



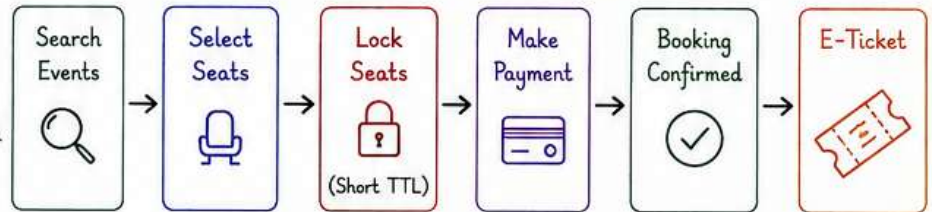
SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 16

32. End-to-End Flow (High Level)

- ① User searches for events.
- ② System shows events & seat map.
- ③ User selects seats.
- ④ System locks the selected seats (short TTL).
- ⑤ User makes payment.
- ⑥ On success, booking is confirmed.
- ⑦ E-Ticket is generated.
- ⑧ Notification (Email / SMS / App).



★ If any step fails, system should release locks & allow retry.

33. Important Design Decisions

Decision	Why
 Microservices	Independent scaling, easier deployment & fault isolation.
 Redis for Seat Lock	Very fast, atomic operations, TTL support.
 Database Partitioning	Handle huge data (events, bookings) efficiently.
 CDN for Static Content	Fast delivery of images, JS, CSS across regions.
 Monitoring & Alerts	Quick detection of issues, better reliability.
 Idempotency Keys	Prevent duplicate bookings & double payments.

34. Handling Failures Gracefully

- ★ Payment failed → Release seat lock.
- ★ Payment timeout → Lock expires, seat released.
- ★ Service down → Retry with exponential backoff.
- ★ Duplicate booking → Prevented via idempotency key.
- ★ Database failure → Use replicas, backups & failover.



Always design for failure, because failures are guaranteed in real systems.

35. Cost Optimization Strategies

- Use Auto-scaling to reduce idle server cost.
- Cache aggressively (Redis, CDN).
- Serverless functions for infrequent tasks.
- Optimize database queries & indexing.
- Use reserved instances for predictable workloads.
- Archive old events & bookings to cheaper storage.



36. Security Best Practices

- ✓ HTTPS everywhere.
- ✓ Secure payment with PCI-DSS compliance.
- ✓ JWT for authentication.
- ✓ Rate limiting to prevent abuse.
- ✓ Input validation & sanitize all inputs.
- ✓ Secrets stored in Vault / Secrets Manager.
- ✓ Regular security audits & penetration testing.



37. Useful Metrics to Track

- Requests per second (RPS)
- Peak concurrent users
- Search success rate
- Seat lock success rate
- Payment success rate
- Booking conversion rate
- Average response time
- Cache hit ratio
- Error rate (%)
- Revenue per event



38. Final Checklist Before Going Live

- ✓ Load testing completed
- ✓ Security validations done
- ✓ Backup & disaster recovery ready
- ✓ Monitoring & alerts configured
- ✓ Seat locking & idempotency tested
- ✓ Auto-scaling configured
- ✓ CDN for static content enabled
- ✓ Database replicas & failover tested
- ✓ Business metrics dashboard ready
- ✓ Runbook for incidents prepared
- ✓ Team on-call rotation setup



Key Takeaway

Ticketmaster is all about handling Massive Scale, ensuring Consistency, providing Great User Experience and staying Reliable under any circumstances. A good design balances Performance, Cost, and Reliability.



Great systems are built by solving hard problems with simple ideas.
Keep learning. Keep building. Keep shipping!



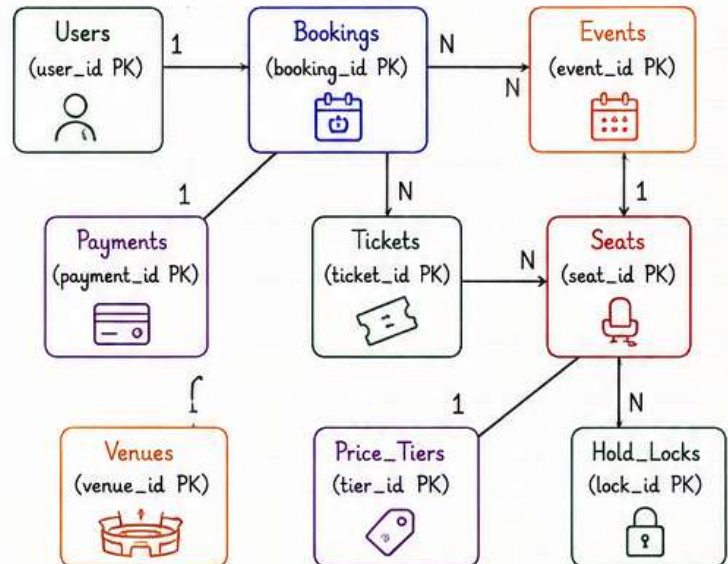
SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 17

39. Database Schema (High Level)

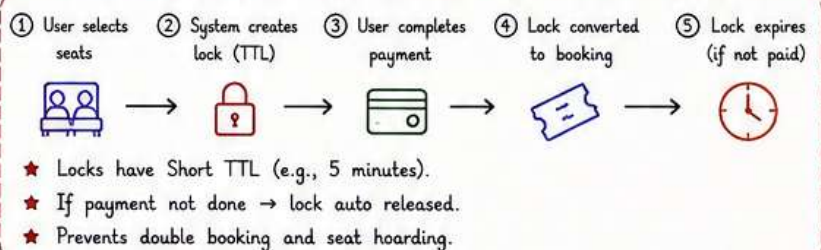
Table	Purpose
Users	Stores user information
Events	Event details (name, venue, time, etc.)
Venues	Venue / stadium information
Seats	Seat layout & section mapping
Bookings	Booking / reservation details
Payments	Payment transactions
Tickets	Generated tickets (QR / Barcodes)
Price_Tiers	Seat categories & pricing
Hold_Locks	Temporary holds on seats
Event_Logs	Activity logs for events & bookings



Key Notes

- ★ Seats belong to a Venue.
- ★ A User can have many Bookings.
- ★ A Booking generates Tickets.
- ★ Hold_Locks temporarily block seats during checkout.

40. Seat Locking Mechanism



41. Caching Plan

- Cache seat inventory for popular events (Redis).
- Cache event details & venue layouts.
- Use CDN for static assets (images, JS, CSS).
- Invalidate cache on booking / seat update.



42. APIs (Examples)

Method	Endpoint	Purpose
GET	/events	List all events
GET	/events/{id}	Get event details & seat map
POST	/seats/lock	Lock selected seats
POST	/bookings	Create booking
GET	/bookings/{id}	Get booking details
POST	/payments	Process payment
GET	/tickets/{id}	Get / download ticket

43. Scalability Tips

- ✓ Auto-scale services based on CPU / traffic.
- ✓ Use Load Balancer & multiple Availability Zones.
- ✓ Database Read Replicas for read-heavy queries.
- ✓ Sharding for huge tables (Bookings, Tickets).
- ✓ Async processing for emails, notifications.
- ✓ Use Message Queue (Kafka) for background jobs.



44. Things to Consider

- ★ Multiple concurrent users trying to book same seats.
- ★ Seat visibility vs hold expiry synchronization.
- ★ Refunds, cancellations & rescheduling.
- ★ International events (timezones, currencies).
- ★ Disaster recovery & data backups.



Final Thought

System Design is not just about making it work,
it's about making it **scale**, **reliable** and **delightful** for millions of users!



Great systems are designed with purpose,
built with care and scaled with confidence.



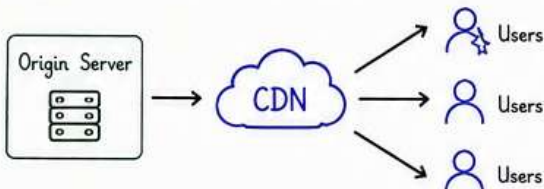
SYSTEM DESIGN HANDBOOK

Design Ticketmaster

Page 18

45. CDN for Static Content

CDN helps deliver static assets (images, JS, CSS) from edge locations for faster load time.



Benefits

- ★ Reduced latency
- ★ Lower bandwidth cost
- ★ Better performance during high traffic
- ★ Offload static content from origin servers

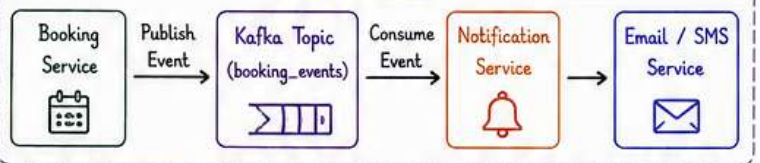


46. Message Queue (Kafka)

- ★ Decouples services and handles async tasks.
- ★ Used for events like Booking Created, Payment Success, Ticket Sent.
- ★ Improves reliability & system resilience.
- ★ Buffers traffic spikes.



Example Flow



Common Topics

- booking_events
- payment_events
- ticket_events
- seat_hold_events
- user_activity_events
- notification_events

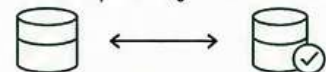


47. Database Choice

MySQL (OLTP) 	Redis (Cache) 	Object Storage (S3 / GCS) 
• Used for transactional data. • Bookings, Payments, Users, Seats, Tickets, etc. • ACID compliance. • Strong consistency.	• Cache seat inventory, venue details, event lists. • Store sessions, temporary locks. • Extremely fast (in-memory). • Reduces DB load.	• Store images, PDFs (tickets), QR codes. • Static files, event posters, seat maps. • Highly durable & scalable.

48. Data Consistency

- ✓ Use DB transactions for critical flows (booking, payment).
- ✓ Use idempotency keys to avoid duplicate bookings.
- ✓ Eventual consistency for non-critical features (analytics, notifications).
- ✓ Use outbox pattern for reliable event publishing.



49. Monitoring & Alerting



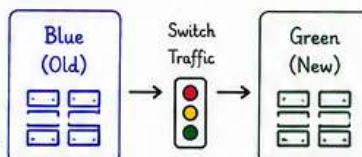
50. Rate Limiting

- ★ Prevent abuse & ensure fairness.
- ★ Limit requests per user / IP.
- ★ Example: 100 requests / minute.
- ★ Implement using Redis (Sliding Window / Token Bucket).



51. Deployment Strategy

- ★ Use Blue-Green or Canary deployment.
- ★ Rolling updates to avoid downtime.
- ★ Automated rollback if error rate increases.
- ★ Use Docker + Kubernetes for container orchestration.



52. Disaster Recovery

- ★ Regular backups (DB & files).
- ★ Multi-region deployment.
- ★ Failover to standby systems.
- ★ Test disaster recovery plan periodically.



Key Takeaways



- ✓ Break the problem into services.
- ✓ Choose the right tech for the right job.
- ✓ Focus on scale, reliability & user experience.
- ✓ Monitor everything.
- ✓ Plan for failure.
- ✓ Keep iterating and improving.



Great systems don't happen by chance,
they are designed, built and scaled with intention.

